Logistic Regression

Formula-Focused Guide with Intuitive Explanations

Machine Intelligence - Andrés Masegosa - Fall 2025

Contents

1 The Big Picture: Why Logistic Regression? 3

1.1 From Regression to Classification . . . 3

2 The Sigmoid Function: The Magic Squasher 5

2.1 The Formula and Its Parts . . . 5

2.2 Understanding Sigmoid Behavior . . . 6

3 Logistic Regression Model 8

3.1 The Complete Model Formula . . . 9

3.2 Making Predictions . . . 10

4 Decision Boundaries 11

4.1 The Math Behind Decision Boundaries . . . 11

4.2 Linear Decision Boundaries . . . 12

4.3 Non-Linear Decision Boundaries . . . 13

5 Cost Function: Measuring Model Quality 15

5.1 Why Not Squared Error? . . . 16

5.2 The Log Loss Function . . . 17

5.3 Total Cost Function . . . 19

6 Gradient Descent for Logistic Regression 21

6.1 The Gradient Formulas . . . 22

6.2 Comparison with Linear Regression . . . 24

6.3 The Complete Algorithm . . . 25

7 Overfitting: The Core Problem 26

7.1 Three Scenarios . . . 28

7.2 Solutions to Overfitting . . . 29

8 Regularization: Controlling Model Complexity 32

8.1 The Regularization Term: Formula Breakdown 34
8.2 Regularized Cost Function 38

9 Regularized Gradient Descent **40**
9.1 The Modified Gradients 41
9.2 Understanding the Extra Term 42
9.3 The Update Rules 44
9.4 Complete Algorithm 45
9.5 Comparison Summary 46

10 Summary: The Complete Picture **47**

1 The Big Picture: Why Logistic Regression?

The Journey: From Numbers to Categories

What We've Learned So Far:

- **Linear Regression**: Predicts continuous numbers (prices, temperatures, distances)
- We learned to find patterns in data and make numerical predictions

The New Challenge:

- Real world often needs YES/NO decisions, not numbers
- Is this email spam or not? Is this tumor malignant or benign?
- We need to classify things into categories

Why This Lecture Exists:

- Logistic regression bridges the gap between numbers and categories
- It's the foundation for understanding neural networks
- It introduces probability thinking to machine learning

The Core Idea: We'll take linear regression (which you know) and transform it to output probabilities (between 0 and 1) instead of any number. This single transformation unlocks classification!

1.1 From Regression to Classification

Two Fundamentally Different Tasks

Regression - Predicting "How Much?":

- Output: Any real number $\in (-\infty, \infty)$
- Example: Predict house price = \$250,000
- Model: $f(\vec{x}) = \vec{w} \cdot \vec{x} + b$ (can be any value)

Classification - Predicting "Which Category?":

- Output: Discrete category (Yes/No, True/False, 0/1)
- Example: Is tumor malignant? $\rightarrow$ Yes (with 87% confidence)
- Model: Must output probability between 0 and 1

The Problem: Linear regression outputs unlimited numbers. Classification needs bounded probabilities. We need a transformation!

Binary Classification Setup

Goal: Classify each input into exactly ONE of TWO classes
Mathematical Notation:

- Output labels: $y \in \{0, 1\}$
- $y = 1$ means "positive class" (yes, true, malignant)
- $y = 0$ means "negative class" (no, false, benign)

What We Want: A model that outputs $P(y = 1|\vec{x})$ - the probability that the class is 1, given input $\vec{x}$

Why Not Use Linear Regression?

Three Fatal Problems:
Problem 1 - Invalid Probabilities:

- Linear regression can output -5 or 50
- Probabilities must be between 0 and 1
- Negative probability makes no sense!

Problem 2 - Outlier Sensitivity:

- One extreme point shifts the entire line
- Classification boundary moves incorrectly
- Unstable predictions

Problem 3 - Wrong Shape:

- Classification needs S-shaped curve (gradual transition)
- Linear function is straight (abrupt changes)
- Doesn't capture probability nature

The Solution: Transform linear output through a function that squashes everything between 0 and 1!

2 The Sigmoid Function: The Magic Squasher

Why Do We Need the Sigmoid Function?

The Core Problem: We have a linear model $z = \vec{w} \cdot \vec{x} + b$ that outputs any number. We need probabilities (0 to 1).
What Properties Do We Need?:

1. Takes ANY real number as input: $z \in (-\infty, \infty)$
2. Outputs ONLY values between 0 and 1: output $\in (0, 1)$
3. Smooth transition (no jumps) - differentiable everywhere
4. Symmetric around 0.5 (fair, unbiased)
5. Output can be interpreted as probability

The Sigmoid Function Does Exactly This! It's not arbitrary - it's the natural solution to transforming unlimited numbers into probabilities.

2.1 The Formula and Its Parts

Sigmoid Function - Dissected

The Formula:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Breaking Down Each Part:
Part 1: e^{-z} (the exponential term)

- When z is large positive: $e^{-z} \approx 0$ (very small)
- When $z = 0$: $e^{-z} = e^0 = 1$
- When z is large negative: e^{-z} becomes very large
- This term controls the "squashing" behavior

Part 2: $1 + e^{-z}$ (the denominator)

- Always ≥ 1 (never zero, so division is always valid)
- When $z \to \infty$: denominator $\to 1$ (since $e^{-z} \to 0$)
- When $z \to -\infty$: denominator $\to \infty$ (since $e^{-z} \to \infty$)

Part 3: $\frac{1}{\text{denominator}}$ (the final output)

- Large denominator $\to$ small output (close to 0)
- Small denominator (close to 1) $\to$ output close to 1
- Always between 0 and 1 (perfect for probability!)

Why This Specific Formula?: The exponential e^{-z} creates smooth, S-shaped curves. The fraction form ensures output stays in $(0, 1)$. This combination is mathematically elegant and has nice derivative properties!

2.2 Understanding Sigmoid Behavior

How Sigmoid Transforms Numbers

Input → Output Mapping:

Input z	e^{-z}	Output $g(z)$	Interpretation
$-\infty$	∞	≈ 0	Very confident: class 0
-5	148	0.007	Strong evidence: class 0
-2	7.39	0.12	Leaning toward: class 0
0	1	0.5	Completely uncertain
2	0.135	0.88	Leaning toward: class 1
5	0.0067	0.993	Strong evidence: class 1
∞	0	≈ 1	Very confident: class 1

The Pattern:

- Large negative z → output near 0 → predict class 0
- z near 0 → output near 0.5 → uncertain, 50-50
- Large positive z → output near 1 → predict class 1

The S-Shape Intuition:

- Starts flat near 0 (for very negative inputs)
- Rapidly rises around $z = 0$ (decision region)
- Flattens again near 1 (for very positive inputs)
- This creates smooth, gradual probability transitions

Computing Sigmoid: Step by Step

Example 1: $z = 2$
Step 1: Compute $e^{-z} = e^{-2}$

- $e^{-2} \approx 0.135$

Step 2: Compute denominator $1 + e^{-2}$

- $1 + 0.135 = 1.135$

Step 3: Divide

- $g(2) = \frac{1}{1.135} \approx 0.88$

Interpretation: 88% probability of class 1

Example 2: $z = -3$
Step 1: $e^{-(-3)} = e^{3} \approx 20.09$
Step 2: $1 + 20.09 = 21.09$
Step 3: $g(-3) = \frac{1}{21.09} \approx 0.047$
Interpretation: Only 4.7% probability of class 1 → Predict class 0

Key Properties of Sigmoid

Property 1 - Range: $g(z) \in (0, 1)$ for all real z

- Never exactly 0 or 1, but can get arbitrarily close
- Always a valid probability

Property 2 - Symmetry: $g(-z) = 1 - g(z)$

- Mirrored around 0.5
- Fair treatment of both classes

Property 3 - Midpoint: $g(0) = 0.5$

- At $z = 0$, model is perfectly uncertain
- Natural decision boundary

Property 4 - Smooth: Differentiable everywhere

- No jumps or kinks
- Can use gradient descent
- Makes optimization possible

3 Logistic Regression Model

Connecting Linear Regression to Classification

The Building Process:
Step 1 - Start with What We Know:

- Linear regression: $z = \vec{w} \cdot \vec{x} + b$
- This captures linear relationships in data
- But outputs unlimited numbers

Step 2 - Apply Sigmoid Transformation:

- Feed z through sigmoid: $g(z) = \frac{1}{1+e^{-z}}$
- Now output is between 0 and 1
- Can interpret as probability!

Step 3 - Logistic Regression is Born:

- $f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b)$
- Linear thinking + Probability transformation
- Best of both worlds!

Why This Works: The linear part $\vec{w} \cdot \vec{x} + b$ learns patterns. The sigmoid converts those patterns into probabilities. Simple but powerful!

3.1 The Complete Model Formula

Logistic Regression - Complete Breakdown

The Full Formula:

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w}\cdot\vec{x}+b)}}$$

Component 1: The Input $\vec{x} = [x_1, x_2, \ldots, x_n]$

- Features describing the data point
- Example: tumor size, patient age, etc.
- This is what we observe/measure

Component 2: The Weights $\vec{w} = [w_1, w_2, \ldots, w_n]$

- Parameters we learn from data
- w_j tells us importance of feature x_j
- Positive $w_j \to$ feature increases probability of class 1
- Negative $w_j \to$ feature decreases probability of class 1
- Large $|w_j| \to$ feature j is very important

Component 3: The Bias b

- Overall offset/baseline probability
- Independent of features
- Shifts decision boundary

Component 4: The Linear Combination $z = \vec{w} \cdot \vec{x} + b$

- Expands to: $z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$
- Weighted sum of features
- This is the "evidence" for class 1
- Can be any real number

Component 5: The Sigmoid Transformation $g(z) = \frac{1}{1+e^{-z}}$

- Converts evidence z into probability
- Large positive $z \to$ high probability
- Large negative $z \to$ low probability

The Output: $f_{\vec{w},b}(\vec{x}) = P(y = 1|\vec{x})$

- Probability that $y = 1$ given input $\vec{x}$
- Always between 0 and 1
- Can use to make binary decision

3.2 Making Predictions

From Probability to Decision

The Three-Step Process:
Step 1: Compute Linear Evidence

$$z = \vec{w} \cdot \vec{x} + b$$

This gives us the "raw score" - positive favors class 1, negative favors class 0
Step 2: Convert to Probability

$$f_{\vec{w},b}(\vec{x}) = g(z) = \frac{1}{1 + e^{-z}}$$

This gives us $P(y = 1|\vec{x})$ - the probability of class 1
Step 3: Make Binary Decision

- If $f_{\vec{w},b}(\vec{x}) \geq 0.5$: Predict $\hat{y} = 1$ (class 1)
- If $f_{\vec{w},b}(\vec{x}) < 0.5$: Predict $\hat{y} = 0$ (class 0)

Why 0.5 Threshold?

- $g(z) = 0.5$ when $z = 0$
- So we predict class 1 when $z \geq 0$, i.e., when $\vec{w} \cdot \vec{x} + b \geq 0$
- This is the natural decision point where evidence is balanced

Complete Example: Tumor Classification

Problem: Predict if tumor is malignant based on size (in cm)
Learned Parameters:

- Weight: $w_1 = 0.5$
- Bias: $b = -5$

Prediction for tumor size = 4 cm:
Step 1 - Compute evidence:

$$z = w_1 \cdot x_1 + b = 0.5 \times 4 + (-5) = 2 - 5 = -3$$

Step 2 - Convert to probability:

$$f_{w,b}(x) = g(-3) = \frac{1}{1 + e^3} = \frac{1}{1 + 20.09} \approx 0.047$$

Step 3 - Make decision:

- $0.047 < 0.5 \rightarrow$ Predict $\hat{y} = 0$ (benign)
- Interpretation: Only 4.7% chance of malignancy

Prediction for tumor size = 12 cm:
Step 1: $z = 0.5 \times 12 - 5 = 6 - 5 = 1$
Step 2: $f(x) = g(1) = \frac{1}{1+e^{-1}} \approx 0.73$
Step 3: $0.73 \geq 0.5 \rightarrow$ Predict $\hat{y} = 1$ (malignant, 73% probability)

4 Decision Boundaries

Why Study Decision Boundaries?

The Visualization Problem: Models make predictions, but how do they "divide" the space of possible inputs?
What is a Decision Boundary?:

- The line/curve/surface that separates regions
- On one side: model predicts class 1
- On other side: model predicts class 0
- On the boundary itself: model is exactly 50-50 uncertain

Why This Matters:

- Helps visualize what model learned
- Shows if model is too simple (straight line when data needs curve)
- Shows if model is too complex (wiggly line fitting noise)
- Guides feature engineering decisions

Decision boundaries are the "shape" of your model's understanding!

4.1 The Math Behind Decision Boundaries

Finding the Decision Boundary

The Logic Chain:
Fact 1: We predict $\hat{y} = 1$ when $f_{\vec{w},b}(\vec{x}) \geq 0.5$
Fact 2: Sigmoid $g(z) = 0.5$ exactly when $z = 0$

- Proof: $g(0) = \frac{1}{1+e^0} = \frac{1}{1+1} = \frac{1}{2} = 0.5$ (correct)

Fact 3: So we predict $\hat{y} = 1$ when $z = \vec{w} \cdot \vec{x} + b \geq 0$
The Decision Boundary is Where Prediction Switches:

$$\boxed{\vec{w} \cdot \vec{x} + b = 0}$$

Breaking This Down:

- Points where $\vec{w} \cdot \vec{x} + b > 0$: Predict class 1
- Points where $\vec{w} \cdot \vec{x} + b < 0$: Predict class 0
- Points where $\vec{w} \cdot \vec{x} + b = 0$: Exactly uncertain (boundary)

Key Insight: The decision boundary is determined ONLY by parameters $\vec{w}$ and b, not by the training data directly!

4.2 Linear Decision Boundaries

2D Case: Two Features

Setup: Features x_1 and x_2, parameters w_1, w_2, b
Model:

$$f_{\vec{w},b}(\vec{x}) = g(w_1x_1 + w_2x_2 + b)$$

Decision Boundary:

$$w_1x_1 + w_2x_2 + b = 0$$

This is the equation of a straight line in 2D!
Example: $w_1 = 1$, $w_2 = 1$, $b = -3$
Decision boundary: $x_1 + x_2 - 3 = 0$
Rearranging: $x_2 = 3 - x_1$
Interpretation:

- Straight line with slope -1, intercept 3
- Above line ($x_1 + x_2 > 3$): Predict class 1
- Below line ($x_1 + x_2 < 3$): Predict class 0

Higher Dimensions

For n Features:

- Feature space is n-dimensional
- Decision boundary: $\sum_{j=1}^{n} w_jx_j + b = 0$
- This is an $(n-1)$-dimensional hyperplane
- Divides space into two half-spaces

Examples:

- 1 feature ($n = 1$): Boundary is a point on number line
- 2 features ($n = 2$): Boundary is a line in plane
- 3 features ($n = 3$): Boundary is a plane in 3D space
- 100 features ($n = 100$): Boundary is 99-dimensional hyperplane

The Linear Limitation: With basic features, decision boundary is always a flat hyperplane. Can't capture curved patterns!

4.3 Non-Linear Decision Boundaries

Why Do We Need Curved Boundaries?

Real World is Not Linear:

- Data often needs circular, elliptical, or complex boundaries
- Example: Tumor malignancy might depend on size AND irregularity in a non-linear way
- Straight lines are too restrictive

The Solution: Create new features that are non-linear combinations of original features!
The Key Insight:

- Logistic regression itself stays "linear in parameters"
- But we transform the input space first
- This lets us fit curves while keeping math tractable

Creating Non-Linear Boundaries

Original Features: x_1, x_2
Add Polynomial Features:

- Quadratic: x_1^2, x_2^2, x_1x_2
- Cubic: $x_1^3, x_2^3, x_1^2x_2, x_1x_2^2$
- Any degree polynomial combinations

Model with Polynomial Features:

$$f_{\vec{w},b}(\vec{x}) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2 + b)$$

What This Creates:

- In original (x_1, x_2) space: Boundary is CURVED
- In expanded feature space: Boundary is still a hyperplane
- We've "lifted" the problem to higher dimensions where it becomes linear!

Example - Circular Boundary:
Model: $f = g(w_3x_1^2 + w_4x_2^2 + b)$ with $w_3 = 1, w_4 = 1, b = -1$
Decision boundary: $x_1^2 + x_2^2 - 1 = 0 \rightarrow x_1^2 + x_2^2 = 1$
This is a circle with radius 1!

- Inside circle: Predict class 1
- Outside circle: Predict class 0

The Power and Peril of Polynomial Features

Benefits:

- Can fit complex, curved decision boundaries
- Captures non-linear relationships
- Makes logistic regression much more flexible

Dangers:

- Too many polynomial features $\rightarrow$ overfitting
- Model becomes too flexible, fits noise
- Decision boundary becomes unreasonably wiggly

Preview: This tension between flexibility and overfitting is WHY we need regularization (coming soon!)

5 Cost Function: Measuring Model Quality

Why Do We Need a Cost Function?

The Training Problem:

- We have training data: $\{(\vec{x}^{(1)}, y^{(1)}), \ldots, (\vec{x}^{(m)}, y^{(m)})\}$
- We have parameters: $\vec{w}, b$ (currently random/unknown)
- **Goal**: Find the BEST values for $\vec{w}, b$

But What Does "Best" Mean?:

- Need a single number measuring how good parameters are
- Low number = good predictions
- High number = bad predictions

The Cost Function Does This:

- Takes parameters $\vec{w}, b$ as input
- Outputs single number: $J(\vec{w}, b)$
- We minimize J to find best parameters

Why Different from Linear Regression?: Squared error works for linear regression but creates non-convex cost for logistic regression. We need a cost function that's convex (single bowl shape) for reliable optimization!

5.1 Why Not Squared Error?

The Squared Error Problem

Linear Regression Cost:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$$

This works great for linear f but...
For Logistic Regression:

- $f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$ is highly non-linear
- Squared error becomes non-convex (multiple local minima)
- Gradient descent might get stuck
- No guarantee of finding global minimum

What is Non-Convex?:

- Imagine a landscape with multiple valleys
- Gradient descent finds nearest valley bottom
- Might miss the deepest valley (global minimum)
- Different starting points give different solutions

We Need: A cost function that creates a single, bowl-shaped landscape (convex)!

5.2 The Log Loss Function

Log Loss for Single Example - Detailed Breakdown

Loss for ONE Training Example $(\vec{x}^{(i)}, y^{(i)})$:

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

Let's call $f_{\vec{w},b}(\vec{x}^{(i)}) = p$ for simplicity (the predicted probability)
Case 1: When Actual Label is $y = 1$
Loss $= -\log(p)$
Understanding $-\log(p)$:

- If model predicts $p = 1$ (100% sure of class 1):
  - Loss $= -\log(1) = 0$ (perfect! No penalty)
- If model predicts $p = 0.5$ (uncertain):
  - Loss $= -\log(0.5) \approx 0.69$ (moderate penalty)
- If model predicts $p = 0.1$ (thinks it's class 0):
  - Loss $= -\log(0.1) \approx 2.3$ (large penalty)
- If model predicts $p \to 0$ (very confident wrong):
  - Loss $\to \infty$ (huge penalty!)

Case 2: When Actual Label is $y = 0$
Loss $= -\log(1 - p)$
Understanding $-\log(1 - p)$:

- If model predicts $p = 0$ (100% sure of class 0):
  - Loss $= -\log(1 - 0) = -\log(1) = 0$ (perfect!)
- If model predicts $p = 0.5$ (uncertain):
  - Loss $= -\log(0.5) \approx 0.69$ (moderate penalty)
- If model predicts $p = 0.9$ (thinks it's class 1):
  - Loss $= -\log(0.1) \approx 2.3$ (large penalty)
- If model predicts $p \to 1$ (very confident wrong):
  - Loss $\to \infty$ (huge penalty!)

Why Logarithm?:

- Grows slowly near correct predictions (gentle penalty for small errors)
- Explodes near confident wrong predictions (harsh penalty for overconfidence)
- Mathematically: relates to maximum likelihood estimation
- Practically: creates convex cost function!

Intuition: Why This Loss Makes Sense

The Penalty Structure:
Correct and Confident → No penalty

- $y = 1, p = 0.99 \rightarrow$ Loss ≈ 0.01 (tiny)
- $y = 0, p = 0.01 \rightarrow$ Loss ≈ 0.01 (tiny)

Correct but Uncertain → Small penalty

- $y = 1, p = 0.7 \rightarrow$ Loss ≈ 0.36 (small)
- Model is right but not confident enough

Wrong but Uncertain → Moderate penalty

- $y = 1, p = 0.4 \rightarrow$ Loss ≈ 0.92 (moderate)
- Model is guessing wrong but hedging

Wrong and Confident → Huge penalty!

- $y = 1, p = 0.01 \rightarrow$ Loss ≈ 4.6 (very large)
- $y = 1, p = 0.001 \rightarrow$ Loss ≈ 6.9 (enormous!)
- Being confidently wrong is heavily penalized

The Philosophy: This loss function rewards calibrated confidence. It's not enough to be right - the model should be appropriately confident when right and uncertain when uncertain!

5.3 Total Cost Function

From Single Loss to Total Cost

Average Loss Over All Examples:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$$

Breaking Down This Formula:
Part 1: $L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$ - Loss for example i

- Measures how wrong prediction is for this single example
- Uses the log loss formula from before

Part 2: $\sum_{i=1}^{m}$ - Sum over all examples

- m is total number of training examples
- Adds up losses from every training example
- Total error across entire dataset

Part 3: $\frac{1}{m}$ - Average

- Divides by number of examples
- Makes cost comparable across different dataset sizes
- "Per example" cost

Expanded Form (Combining Both Cases):

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right]$$

Why This Compact Form Works:

- When $y^{(i)} = 1$:
  - First term: $1 \cdot \log(f) = \log(f)$ (active)
  - Second term: $(1 - 1) \cdot \log(1 - f) = 0$ (vanishes)
  - Result: $-\log(f)$ (correct)
- When $y^{(i)} = 0$:
  - First term: $0 \cdot \log(f) = 0$ (vanishes)
  - Second term: $(1 - 0) \cdot \log(1 - f) = \log(1 - f)$ (active)
  - Result: $-\log(1 - f)$ (correct)

This is a clever mathematical trick to write both cases in one formula!

Cost Function: Key Properties

Property 1 - Convexity:

- $J(\vec{w}, b)$ has single bowl shape
- No local minima to get stuck in
- Gradient descent guaranteed to find global minimum

Property 2 - Differentiability:

- Smooth everywhere (no kinks or jumps)
- Can compute gradients
- Enables gradient descent optimization

Property 3 - Interpretation:

- Low $J \rightarrow$ Model predictions match true labels
- High $J \rightarrow$ Model predictions don't match labels
- Minimum $J \rightarrow$ Best possible parameters

Property 4 - Scale:

- Cost is always non-negative: $J \geq 0$
- Perfect predictions give $J = 0$
- Terrible predictions give large J

Name: Also called **Binary Cross-Entropy Loss** or **Log Loss**

6 Gradient Descent for Logistic Regression

Why Do We Need Gradient Descent?

The Optimization Problem:

- We have cost function $J(\vec{w}, b)$ measuring parameter quality
- **Goal**: Find $\vec{w}$ and b that minimize J
- Problem: Can't solve analytically (no closed-form solution)

The Gradient Descent Solution:

- Start with random parameters
- Iteratively take small steps downhill
- Follow the steepest descent direction (negative gradient)
- Eventually reach the minimum

Why It Works: Cost function is convex (bowl-shaped), so any downhill path leads to global minimum!

6.1 The Gradient Formulas

Gradient Descent Update Rules - Detailed

The Update Algorithm:

$$\text{repeat until convergence}\{$$
$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \quad \text{for } j = 1, \ldots, n$$
$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$
$$\}$$

The Gradient Formulas:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

where $f_{\vec{w},b}(\vec{x}^{(i)}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}^{(i)}+b)}}$

Breaking Down the Weight Gradient:

Part 1: $\frac{1}{m}$ - Average over all examples

- Makes gradient scale-independent of dataset size
- "Per example" gradient

Part 2: $(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$ - Prediction error

- Difference between predicted and actual
- Positive when overpredicting, negative when underpredicting
- Large error $\rightarrow$ large gradient $\rightarrow$ big update

Part 3: $x_j^{(i)}$ - Feature value

- Weights the error by feature magnitude
- If $x_j^{(i)}$ is large, error affects w_j more
- Connects error to specific feature

Part 4: $\sum_{i=1}^{m}$ - Sum contributions

- Each training example "votes" on gradient direction
- More examples pointing same way $\rightarrow$ stronger signal

The Update: $w_j = w_j - \alpha \times \text{gradient}$

- α is learning rate (step size)
- Negative sign means go opposite to gradient (downhill)
- Moves w_j to reduce cost

Understanding the Gradient Intuitively

What Does Gradient Tell Us?
Positive Gradient:

- $\frac{\partial J}{\partial w_j} > 0$
- Increasing w_j would increase cost
- So we *decrease* w_j (note the minus sign)

Negative Gradient:

- $\frac{\partial J}{\partial w_j} < 0$
- Increasing w_j would decrease cost
- So we *increase* w_j

Zero Gradient:

- $\frac{\partial J}{\partial w_j} = 0$
- We're at a minimum (or maximum/saddle point)
- Don't update w_j (already optimal)

Magnitude of Gradient:

- Large |gradient| $\rightarrow$ steep slope $\rightarrow$ big updates
- Small |gradient| $\rightarrow$ gentle slope $\rightarrow$ small updates
- Gradient naturally decays as we approach minimum

The Learning Rate α:

- Too small $\rightarrow$ slow convergence (many iterations)
- Too large $\rightarrow$ might overshoot minimum
- Just right $\rightarrow$ efficient convergence
- Typical values: $0.001, 0.01, 0.1, 1$

6.2 Comparison with Linear Regression

Surprising Similarity!

Linear Regression Gradient Descent:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$
$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Logistic Regression Gradient Descent:

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$
$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

The Formulas Look IDENTICAL!
The Key Difference:

- Different definition of $f_{\vec{w},b}(\vec{x})$
- Linear: f is just linear combination
- Logistic: f includes sigmoid transformation

Why They Look the Same: Mathematical coincidence! The derivatives of squared error (linear regression) and log loss (logistic regression) have the same form. But the underlying models are different, so the updates behave differently!

6.3 The Complete Algorithm

Gradient Descent: Step by Step

Initialization:

- Set $w_1, w_2, \ldots, w_n, b$ to small random values or zeros
- Choose learning rate α (e.g., 0.01)
- Set maximum iterations (e.g., 10000)

Repeat Until Convergence:
Step 1 - Compute Predictions: For each training example $i = 1, \ldots, m$:

$$z^{(i)} = \vec{w} \cdot \vec{x}^{(i)} + b$$

$$f_{\vec{w},b}(\vec{x}^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

Step 2 - Compute Gradients: For each parameter w_j where $j = 1, \ldots, n$:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

For bias b:

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

Step 3 - Update Parameters (Simultaneously!):

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j} \quad \text{for all } j$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

CRITICAL: Use temporary variables to update all parameters at once, not one at a time!
Step 4 - Check Convergence:

- Compute cost $J(\vec{w}, b)$
- If change in J is very small, stop
- Or if maximum iterations reached, stop

Output: Optimized parameters $\vec{w}$ and b

7 Overfitting: The Core Problem

Why Does Overfitting Matter?

The Goal of Machine Learning:

- NOT to memorize training data
- But to learn patterns that generalize to NEW data
- Model should perform well on unseen examples

The Overfitting Problem:

- Model fits training data TOO well
- Learns noise and random fluctuations
- Poor performance on new data
- High training accuracy, low test accuracy

Why This is Dangerous:

- Model looks great during training
- Fails catastrophically in real world
- Can't be trusted for deployment

This section explains how to detect and prevent overfitting!

7.1 Three Scenarios

The Three Fates of a Classification Model

Scenario 1: Underfitting (High Bias)
What It Is:

- Model too simple for the data complexity
- Can't capture underlying patterns
- Poor performance everywhere

Symptoms:

- High training error (e.g., 40%)
- High test error (e.g., 45%)
- Decision boundary too simple (straight line for circular data)
- Misclassifies many training points

Example: Using linear boundary when data needs curves

Scenario 2: Good Fit (Just Right)
What It Is:

- Model complexity matches problem complexity
- Captures true patterns without noise
- Generalizes well

Symptoms:

- Low training error (e.g., 5%)
- Low test error (e.g., 8%)
- Small gap between training and test performance
- Smooth, reasonable decision boundary

Example: Appropriate polynomial degree for data

Scenario 3: Overfitting (High Variance)
What It Is:

- Model too complex for available data
- Memorizes training examples
- Fails to generalize

Symptoms:

- Very low training error (e.g., 0.1%)
- High test error (e.g., 35%)
- Large gap between training and test
- Extremely wiggly decision boundary

Example: Very high-degree polynomial with few training examples

Causes of Overfitting

Cause 1: Too Many Features

- Model has too many parameters
- Can fit any pattern, including noise
- Example: 100 polynomial features for 50 training examples

Cause 2: Too Few Training Examples

- Not enough data to constrain model
- Model finds patterns in random noise
- Example: 10 training points with 20 features

Cause 3: Model Too Complex for Problem

- Using high-degree polynomials for simple linear relationship
- Model has unnecessary flexibility
- Complexity exceeds problem requirements

The Telltale Sign: Training accuracy near 100%, test accuracy much lower $\rightarrow$ Overfitting!

7.2 Solutions to Overfitting

Solution 1: Collect More Data

How It Helps:

- More training examples constrain the model
- Noise averages out with larger datasets
- Model forced to learn general patterns
- Can't memorize everything

When to Use:

- Additional data is available or can be collected
- Cost of data collection is reasonable
- Dataset currently small relative to features

Limitations:

- Data may be expensive or impossible to get
- Doesn't fix fundamental model design issues
- May still overfit with very complex models

Rule of Thumb: Need roughly 10 times as many examples as features for stable learning

Solution 2: Feature Selection

The Idea: Reduce number of features - keep only the most relevant ones
Methods:

- **Manual selection**: Use domain knowledge to pick features
- **Forward selection**: Start with nothing, add features one by one
- **Backward elimination**: Start with all, remove worst feature repeatedly
- **Feature importance**: Rank features by predictive power

Benefits:

- Simpler model, less overfitting
- Faster training and prediction
- More interpretable

Drawbacks:

- Might discard useful information
- Time-consuming process
- Hard to know which features to keep

Example: From 100 features, keep only top 15 most predictive ones

Solution 3: Regularization

The Core Idea: Keep ALL features but penalize large parameter values
How It Works:

- Add penalty term to cost function
- Penalty grows with parameter magnitude
- Model "pays" for using large weights
- Forces model to use features economically

Benefits:

- Keeps all features (no information loss)
- Automatically determines feature importance
- Unimportant features get weights near zero
- Important features keep significant weights

Why It's Best:

- No need to manually select features
- Smooth control over model complexity
- Works well when many features contribute weakly
- Most commonly used in practice

Preview: This is what we'll focus on next!

8 Regularization: Controlling Model Complexity

The Philosophy of Regularization

The Fundamental Trade-off:

- **Flexibility**: Model can fit complex patterns (good)
- **Overfitting**: Too much flexibility fits noise (bad)

The Regularization Insight:

- Don't eliminate features (might need them)
- Instead, make model "pay" for complexity
- Complexity = large parameter values
- Model must balance fitting data vs keeping parameters small

The Analogy - Budget Constraint:

- **Without regularization**: Unlimited budget, model can use huge weights
- **With regularization**: Limited budget, model must spend wisely
- Forces model to prioritize important features

Why Large Weights are Bad:

- Large weight $\rightarrow$ feature has huge impact
- Small data fluctuation causes big prediction change
- Model becomes sensitive to noise
- Small, distributed weights are more robust

The Goal: Encourage simple models while retaining flexibility to fit true patterns!

8.1 The Regularization Term: Formula Breakdown

Understanding the Regularization Term

The Term:

$$\text{Regularization Term} = \frac{\lambda}{2m} \sum_{j=0}^{n-1} w_j^2$$

Let's dissect every single piece...
Part 1: w_j^2 - Squared Weight
Why square the weight?:

- Treats positive and negative weights equally: $(-3)^2 = 3^2 = 9$
- Quadratic penalty: penalty grows faster than weight magnitude
- Small weight $w = 0.1$: penalty $= 0.01$ (very small)
- Medium weight $w = 1$: penalty $= 1$ (moderate)
- Large weight $w = 10$: penalty $= 100$ (huge!)

Why not $|w_j|$?:

- Could use absolute value (called L1 regularization)
- Square is differentiable everywhere (better for gradient descent)
- Square creates "smooth" penalty landscape
- This is L2 regularization (most common)

Part 2: $\sum_{j=0}^{n-1} w_j^2$ - Sum of Squared Weights
What this measures:

- Total "magnitude" of all weights
- Also written as $\|\vec{w}\|^2$ (L2 norm squared)
- Captures overall model complexity

Why sum all weights?:

- Want to penalize total complexity, not just one weight
- Model might spread complexity across many weights
- Sum catches all forms of complexity

Example:

- Model A: $w_1 = 10, w_2 = 0 \rightarrow$ penalty $= 100 + 0 = 100$
- Model B: $w_1 = 1, w_2 = 1 \rightarrow$ penalty $= 1 + 1 = 2$
- Model B preferred (distributes weights smoothly)

Part 3: $\frac{1}{2m}$ - Scaling Factor
The m in denominator:

- m = number of training examples
- Makes penalty scale-independent of dataset size
- Ensures λ means same thing for different datasets

Why?:

What Regularization Does Intuitively

Without Regularization:
Model's only goal: Minimize prediction error

$$\min_{\vec{w},b} \{\text{Prediction Error}\}$$

- Can achieve zero training error with complex model
- No constraint on weight values
- Leads to overfitting

With Regularization:
Model has two competing goals:

$$\min_{\vec{w},b} \{\text{Prediction Error} + \lambda \times \text{Weight Size}\}$$

- Goal 1: Fit training data (minimize error)
- Goal 2: Keep weights small (minimize complexity)
- Must balance both goals
- Can't perfectly fit training data if it requires huge weights
- Forced to find simpler, more generalizable solution

The Balancing Act:

- λ controls the balance
- Small λ: Prioritize fitting data
- Large λ: Prioritize small weights
- Optimal λ: Best generalization to new data

Key Point: Bias Not Regularized

Notice: We DON'T include b in regularization term
Why not regularize bias b?:
Reason 1 - Role of Bias:

- Bias shifts model up/down (or moves decision boundary)
- Doesn't affect model complexity
- Doesn't interact with features
- Just sets baseline prediction

Reason 2 - Mathematical Justification:

- Large weights $\rightarrow$ sensitive to feature noise $\rightarrow$ overfitting
- Large bias $\rightarrow$ just shifts predictions $\rightarrow$ no overfitting
- Bias doesn't amplify noise

Reason 3 - Practical Impact:

- Including b^2 in penalty has negligible effect
- Makes almost no difference to results
- Simpler to exclude it

Convention: Only regularize weights $w_1, w_2, \ldots, w_n$, NOT bias b

8.2 Regularized Cost Function

Complete Regularized Cost Function

The Full Formula:

$$J(\vec{w}, b) = \underbrace{-\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f_{\vec{w},b}(\vec{x}^{(i)}))\right]}_{\text{Loss Term (Prediction Accuracy)}} + \underbrace{\frac{\lambda}{2m}\sum_{j=0}^{n-1} w_j^2}_{\text{Regularization Term (Simplicity)}}$$

Two Components Explained:
Component 1: Loss Term

$$-\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f_{\vec{w},b}(\vec{x}^{(i)}))\right]$$

What it measures:

- How well model fits training data
- Binary cross-entropy loss
- Lower = better predictions

What model wants:

- Minimize this term
- Make predictions match labels
- Achieve low training error

Component 2: Regularization Term

$$\frac{\lambda}{2m}\sum_{j=0}^{n-1} w_j^2$$

What it measures:

- Total magnitude of weights
- Model complexity
- Lower = simpler model

What model wants:

- Minimize this term too
- Keep weights small
- Maintain simplicity

The Trade-off:

- Can't minimize both perfectly
- Reducing loss might require large weights
- Keeping weights small might increase loss
- Must find optimal compromise

The Balancing Act:

How Regularization Prevents Overfitting - The Mechanism

Without Regularization - The Overfit Path:

1. Model sees training data
2. Wants zero training error
3. Uses large weights to fit every detail
4. Fits noise and outliers perfectly
5. Decision boundary becomes extremely wiggly
6. Result: Great training performance, poor test performance

With Regularization - The Balanced Path:

1. Model sees training data
2. Wants low error BUT also wants small weights
3. Can't use arbitrarily large weights (penalty too high)
4. Must ignore some training details (noise)
5. Decision boundary stays reasonably smooth
6. Result: Good training performance, good test performance

Why This Works:

- True patterns usually don't require extreme weights
- Noise requires extreme weights to fit
- By penalizing large weights, we penalize noise-fitting
- True patterns emerge naturally with moderate weights

The Key Insight: Regularization doesn't know what's noise and what's signal. It just discourages complexity. True patterns survive this constraint; noise doesn't!

9 Regularized Gradient Descent

Why Do Gradients Change with Regularization?

Cost Function Changed:

- Added regularization term: $\frac{\lambda}{2m} \sum w_j^2$
- Gradients depend on cost function
- So gradients must change too

What Changes:

- Weight gradient gets extra term
- Bias gradient stays the same (not regularized)

The Effect:

- Weights pulled toward zero each iteration
- Called "weight decay"
- Automatic simplification during training

9.1 The Modified Gradients

Regularized Gradients - Complete Derivation

Cost Function:

$$J(\vec{w}, b) = \underbrace{-\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f_{\vec{w},b}(\vec{x}^{(i)}))\right]}_{\text{Loss } L(\vec{w},b)} + \underbrace{\frac{\lambda}{2m}\sum_{j=0}^{n-1} w_j^2}_{\text{Reg } R(\vec{w})}$$

Gradient of Weight w_j:

$$\frac{\partial J}{\partial w_j} = \frac{\partial L}{\partial w_j} + \frac{\partial R}{\partial w_j}$$

Part 1: Loss Gradient (Same as Before):

$$\frac{\partial L}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)}$$

This is the standard logistic regression gradient - measures prediction error contribution
Part 2: Regularization Gradient (NEW!):

$$\frac{\partial R}{\partial w_j} = \frac{\partial}{\partial w_j}\left(\frac{\lambda}{2m}\sum_{k=0}^{n-1} w_k^2\right)$$

Breaking down the derivative:

- Sum over all weights: $\sum_{k=0}^{n-1} w_k^2 = w_0^2 + w_1^2 + \cdots + w_{n-1}^2$
- When taking $\frac{\partial}{\partial w_j}$, only w_j^2 term survives
- All other w_k^2 terms have zero derivative with respect to w_j

Computing:

$$\frac{\partial}{\partial w_j}(w_j^2) = 2w_j$$

$$\frac{\partial R}{\partial w_j} = \frac{\lambda}{2m}\cdot 2w_j = \frac{\lambda}{m}w_j$$

Combined Gradient for Weight:

$$\boxed{\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}w_j}$$

Gradient for Bias (Unchanged):

$$\boxed{\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})}$$

No regularization term because b is not regularized!

9.2 Understanding the Extra Term

What Does $\frac{\lambda}{m} w_j$ Do?

The Weight Gradient:

$$\frac{\partial J}{\partial w_j} = \underbrace{\frac{1}{m}\sum_{i=1}^{m}(f - y^{(i)})x_j^{(i)}}_{\text{Error-based gradient}} + \underbrace{\frac{\lambda}{m} w_j}_{\text{Regularization gradient}}$$

First Term - Error-Based:

- Depends on training data errors
- Points toward better data fit
- Large when predictions are wrong
- Different for each training example

Second Term - Regularization:

- Depends ONLY on current weight w_j
- Always proportional to w_j itself
- Positive when $w_j > 0$ (push weight down)
- Negative when $w_j < 0$ (push weight up)
- **Always pushes weight toward zero!**

The Tug-of-War:

- Error term: "Increase w_j to better fit training data!"
- Regularization term: "Decrease w_j to keep model simple!"
- Final gradient: Compromise between both forces
- λ controls which force wins

Magnitude Matters:

- Small $|w_j|$: Regularization term small, data fitting dominates
- Large $|w_j|$: Regularization term large, shrinkage dominates
- Self-regulating mechanism!

9.3 The Update Rules

Regularized Gradient Descent Update Rules

Standard Form:

repeat until convergence{

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \right]$$

}

Alternative Form (Revealing Weight Decay):
Let's expand the weight update:

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f - y^{(i)}) x_j^{(i)} \right] - \alpha \frac{\lambda}{m} w_j$$

Rearranging:

$$w_j = w_j - \alpha \frac{\lambda}{m} w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f - y^{(i)}) x_j^{(i)} \right]$$

Factor out w_j:

$$w_j = w_j \left(1 - \alpha \frac{\lambda}{m} \right) - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f - y^{(i)}) x_j^{(i)} \right]$$

Interpretation:
Step 1 - Decay: $w_j \leftarrow w_j \left(1 - \alpha \frac{\lambda}{m}\right)$

- Multiply w_j by factor < 1
- Example: If $\alpha = 0.1, \lambda = 10, m = 100$:
  - Factor $= 1 - 0.1 \times \frac{10}{100} = 1 - 0.01 = 0.99$
- Weight shrinks by 1% each iteration
- This is the "weight decay"!

Step 2 - Update: Subtract error-based gradient

- Same as non-regularized gradient descent
- Adjusts weight to fit data better

Every Iteration:

1. Shrink all weights slightly (decay)
2. Then update based on training error
3. Weights can grow back if needed for data fitting
4. But must overcome the decay force

Why "Weight Decay"?: Weights naturally decay toward zero unless justified by training data!

9.4 Complete Algorithm

Full Regularized Gradient Descent Algorithm

Input:

- Training data: $\{(\vec{x}^{(1)}, y^{(1)}), \ldots, (\vec{x}^{(m)}, y^{(m)})\}$
- Learning rate: α (e.g., 0.01)
- Regularization parameter: λ (e.g., 1)
- Max iterations: N (e.g., 10000)

Initialize:

- $w_1, w_2, \ldots, w_n \leftarrow 0$ (or small random values)
- $b \leftarrow 0$

For $iter = 1$ to N:

Step 1 - Compute Predictions:
For each example $i = 1, \ldots, m$:

$$f_{\vec{w},b}(\vec{x}^{(i)}) = \frac{1}{1 + e^{-(\vec{w}\cdot\vec{x}^{(i)}+b)}}$$

Step 2 - Compute Gradients:
For each weight $j = 1, \ldots, n$:

$$\text{grad}_{w_j} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}w_j$$

For bias:

$$\text{grad}_b = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

Step 3 - Update Parameters (Simultaneously):
For all j:

$$w_j \leftarrow w_j - \alpha \cdot \text{grad}_{w_j}$$

For bias:

$$b \leftarrow b - \alpha \cdot \text{grad}_b$$

Step 4 - Check Convergence (Optional):
Compute cost $J(\vec{w}, b)$
If change in J is very small (e.g., < 0.0001), stop

Output: Learned parameters $\vec{w}$ and b

9.5 Comparison Summary

With vs Without Regularization

Without Regularization:
Cost:

$$J = \frac{1}{m} \sum \text{Loss}$$

Weight gradient:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum (f - y^{(i)}) x_j^{(i)}$$

Update:

$$w_j \leftarrow w_j - \alpha \cdot \text{gradient}$$

Behavior:

- Minimizes training error only
- No constraint on weights
- Can overfit with many features

With Regularization:
Cost:

$$J = \frac{1}{m} \sum \text{Loss} + \frac{\lambda}{2m} \sum w_j^2$$

Weight gradient:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum (f - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j$$

Update:

$$w_j \leftarrow w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \cdot \text{error gradient}$$

Behavior:

- Balances training error and weight magnitude
- Weights decay toward zero
- Prevents overfitting
- Better generalization

Key Difference: The $\frac{\lambda}{m} w_j$ term in weight gradient!

10 Summary: The Complete Picture

Essential Formulas - Master These

1. Sigmoid Function:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Transforms any number to probability between 0 and 1

2. Logistic Regression Model:

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

Output $= P(y = 1|\vec{x})$

3. Decision Boundary:

$$\vec{w} \cdot \vec{x} + b = 0$$

Predict $\hat{y} = 1$ if $\vec{w} \cdot \vec{x} + b \geq 0$, else $\hat{y} = 0$

4. Cost Function (Binary Cross-Entropy):

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f) + (1 - y^{(i)}) \log(1 - f) \right]$$

where $f = f_{\vec{w},b}(\vec{x}^{(i)})$

5. Regularized Cost Function:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f) + (1 - y^{(i)}) \log(1 - f) \right] + \frac{\lambda}{2m} \sum_{j=0}^{n-1} w_j^2$$

6. Gradients (Without Regularization):

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (f - y^{(i)})$$

7. Gradients (With Regularization):

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (f - y^{(i)})$$

8. Update Rules:

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$

$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

Conceptual Summary

What We Learned:
The Problem: Need to classify data into categories, but linear regression doesn't work
The Solution: Logistic regression

- Start with linear model: $z = \vec{w} \cdot \vec{x} + b$
- Transform through sigmoid: $g(z) = \frac{1}{1+e^{-z}}$
- Output is probability: $P(y = 1|\vec{x})$

Training the Model:

- Cost function: Binary cross-entropy (log loss)
- Optimization: Gradient descent
- Same update formula structure as linear regression
- But different model (includes sigmoid)

The Overfitting Problem:

- Model too complex $\rightarrow$ memorizes training data
- Poor performance on new data
- Telltale sign: Large gap between training and test accuracy

The Solution - Regularization:

- Add penalty for large weights: $\frac{\lambda}{2m} \sum w_j^2$
- Balances fitting data vs keeping model simple
- λ controls the trade-off
- Prevents overfitting without losing features

Why Regularization Works:

- Large weights $\rightarrow$ complex model $\rightarrow$ fits noise
- Small weights $\rightarrow$ simple model $\rightarrow$ captures true patterns
- Regularization encourages small weights
- True patterns survive, noise doesn't

The Big Picture: Why Logistic Regression Matters

Foundation for Advanced Topics:

- **Neural Networks**: Logistic units stacked in layers
- **Deep Learning**: Many layers of logistic transformations
- **Modern AI**: Built on these same principles

Key Concepts Introduced:

- Probability-based predictions
- Non-linear transformations (sigmoid)
- Classification vs regression
- Overfitting and regularization
- Trade-off between bias and variance

Skills Developed:

- Understanding cost functions
- Deriving and interpreting gradients
- Balancing model complexity
- Thinking about generalization

What Makes This Important: Logistic regression is simple enough to understand deeply, yet powerful enough to solve real problems. It teaches fundamental concepts that apply to all of machine learning!

Practical Advice

When Using Logistic Regression:

1. **Feature Scaling**:
   - Standardize features (z-score normalization)
   - Makes gradient descent converge faster
   - Prevents features with large values from dominating
2. **Learning Rate Selection**:
   - Start with $\alpha = 0.01$ or 0.1
   - If cost oscillates: reduce α
   - If convergence too slow: increase α
   - Monitor cost function - should decrease monotonically
3. **Regularization Parameter**:
   - Try $\lambda \in \{0.01, 0.1, 1, 10, 100\}$
   - Use validation set to select best λ
   - Too small $\rightarrow$ overfitting
   - Too large $\rightarrow$ underfitting
4. **Convergence Monitoring**:
   - Plot cost vs iterations
   - Should decrease smoothly
   - Stop when change is very small
   - Or use fixed number of iterations
5. **Evaluation**:
   - ALWAYS use separate test set
   - Training accuracy can be misleading
   - Look for train-test gap (overfitting indicator)
   - Consider precision, recall, F1 score
6. **Feature Engineering**:
   - Start with linear model
   - Add polynomial features if needed
   - Use regularization to prevent overfitting
   - Don't add too many features without more data